Studio Nx

JS 29 — formatRelativeTime()

2026-10-12

JS 29 — `formatRelativeTime()`

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면: 목록의 "마지막 수정" 칸 글자 (PRD 01 FR-123)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	28 `initContactForm()` · 30 `loadProjects()`

1. 하는 일

서버가 준 수정 시각(`updated_at`)을 사람이 읽기 쉬운 글자로 바꿉니다. 목록 오른쪽의 "언제" 칸에 들어가는 글자입니다 (HTML 예시와 같은 모양).

수정한 지	글자
1분 안 (또는 미래: 컴퓨터 시계가 조금 느릴 때)	`Just now`
1분, 12분	`1 minute ago`, `12 minutes ago`
1시간, 11시간 (24시간 안)	`1 hour ago`, `11 hours ago`
달력으로 어제	`Yesterday`
2~7일 전	`2 days ago` … `7 days ago`
8일 이상이고 올해	`Sep 30`
다른 해	`Dec 31, 2025`

2. 언제 쓰나

호출하는 곳	화면
`renderProjects` (31, 다음 함수들)	목록 한 줄의 `[data-field="when"]` 칸입니다. 예: `Site Song … 12 minutes ago`

서버 시각의 모양: 서버는 UTC 시각을 **시간대 표시 없이** 보냅니다 (`"2026-10-08T02:10:11.123000"`). 자바스크립트의 `new Date()`는 시간대 표시가 없는 날짜·시각 문자열을 **이 컴퓨터의 시간대**로 읽으므로, 그대로 쓰면 서울에서는 9시간이 어긋납니다. 그래서 끝에 `Z`(UTC라는 표시)를 붙여서 읽습니다. userInfo의 가입일(`formatDate`, P1-1022)과 같은 방법입니다.

3. 코드

import (`myproject.js` 맨 위)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout } from '../core/auth.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio } from '../core/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

이 함수는 import를 쓰지 않습니다. 날짜 계산은 브라우저의 기본 기능(`Date`, `toLocaleDateString`)만으로 합니다. (import 정리는 다음 함수들에서 합니다.)

함수

```
export function formatRelativeTime(isoTime, now = new Date()) {
  const hasZone = /[zZ]$|[+-]\d\d:\d\d$/.test(isoTime);
  const time = new Date(hasZone ? isoTime : isoTime + 'Z');
  const minutes = Math.floor((now - time) / 60000);

  if (minutes < 1) return 'Just now';
  if (minutes < 60) return minutes === 1 ? '1 minute ago' : minutes + ' minutes ago';
  const hours = Math.floor(minutes / 60);
  if (hours < 24) return hours === 1 ? '1 hour ago' : hours + ' hours ago';

  // 달력으로 며칠 전인지 — 두 날의 자정끼리 비교 (시각은 버림)
  const today = new Date(now.getFullYear(), now.getMonth(), now.getDate());
  const day = new Date(time.getFullYear(), time.getMonth(), time.getDate());
  const days = Math.round((today - day) / 86400000);   // round: 서머타임으로 하루가 23·25시간인 날 대비
  if (days <= 1) return 'Yesterday';
  if (days <= 7) return days + ' days ago';

  const options = { month: 'short', day: 'numeric' };
  if (time.getFullYear() !== now.getFullYear()) options.year = 'numeric';
  return time.toLocaleDateString('en-US', options);
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
if (minutes < 60) return `${minutes} minute${minutes === 1 ? '' : 's'} ago`;
…
return time.toLocaleDateString('en-US', {
  month: 'short', day: 'numeric',
  ...(time.getFullYear() !== now.getFullYear() && { year: 'numeric' }),
});
```

최신 문법	실제 코드	뜻
`` `${minutes} minute…` `` (템플릿 문자열)	`minutes + ' minutes ago'`	문자열 안에 값을 넣습니다
`...(조건 && { year: 'numeric' })`	`if (조건) options.year = 'numeric'`	조건이 맞을 때만 칸을 추가합니다

문법 풀이

import 문법	뜻
(이 함수는 import 없음)	`Date`, `Math`, `toLocaleDateString`은 브라우저에 원래 있습니다

식	뜻	예
`now = new Date()` (매개변수 기본값)	넘기지 않으면 지금 시각입니다. 시험할 때만 시각을 넘깁니다	`f(iso)`는 지금 기준, `f(iso, 정오)`는 정오 기준
`/[zZ]$\|[+-]\d\d:\d\d$/.test(isoTime)`	끝이 `Z` 또는 `+09:00`처럼 시간대로 끝나는지 확인합니다	`"...02:48:00"` → false, `"...02:48:00Z"` → true
`now - time`	두 `Date`를 빼면 **밀리초** 차이가 나옵니다	12분 = 720000

식	뜻	예
`/ 60000`, `Math.floor`	밀리초를 분으로 바꾸고, 소수점을 버립니다	11분 59.9초 → 11
`new Date(년, 월, 일)`	그날 자정입니다 (이 컴퓨터의 시간대 기준)	시각을 버리고 날짜만 비교하려고 씁니다
`/ 86400000`, `Math.round`	밀리초를 일로 바꾸고(하루 = 86 400 000ms), 반올림합니다	서머타임이 바뀌는 날의 3.04 → 3
`toLocaleDateString('en-US', { month: 'short', day: 'numeric' })`	영어 날짜 글자를 만듭니다	`Sep 30`이고, year를 더하면 `Dec 31, 2025`

4. 흐름

1. 시간대 표시가 없으면 `Z`를 붙여 UTC로 읽습니다.
2. 지금과의 차이(분)를 구해서 1분 안, 60분 안, 24시간 안이면 그에 맞는 글자를 돌려줍니다.
3. 24시간이 넘으면 **달력** 기준으로 며칠 전인지 세서 어제, 7일 안, 그 밖에는 날짜를 돌려줍니다.

5. 왜 이렇게

선택	이유
`Z`를 붙여서 읽음	서버 시각은 UTC인데 표시가 없습니다. 붙이지 않으면 서울에서는 "9 hours ago"가 "Just now"로 나오는 식으로 어긋납니다
이미 시간대가 있으면 그대로 씀	나중에 서버가 `Z`를 붙여 보내더라도 `ZZ`가 되어 깨지지 않게 합니다
24시간 안은 **시간**으로, 넘으면 **달력**으로 셈	밤 12시 30분에 본 어젯밤 11시의 수정은 "Yesterday"보다 "1 hour ago"가 정확합니다. 하루가 넘으면 "어제, 3일 전"이 사람이 생각하는 방식입니다
미래 시각은 `Just now`	컴퓨터 시계가 서버보다 조금 느리면 방금 만든 프로젝트가 "-1 minutes ago"로 나올 수 있습니다
일 수는 `Math.round`	서머타임이 바뀌는 날은 하루가 23시간이나 25시간이므로 나누면 2.96이나 3.04가 나옵니다. 미국과 유럽 사용자에 대비한 것입니다
7일이 넘으면 날짜를, 다른 해면 연도까지 보여 줌	템플릿과 HTML 예시와 같습니다. "작년 12월 31일"을 "Dec 31"로만 쓰면 올해로 착각할 수 있습니다
`now` 매개변수	시험에서 "지금"을 고정하려고 둡니다. 화면에서는 넘기지 않습니다
`'en-US'`로 고정	시켠서 밖의 페이지는 영어만 씁니다 (2026-10-07 결정)

6. 시험 (실제 myProject.html, 헤드리스 Chrome, 20개)

페이지 안에서 `import('/static/js/pages/myproject.js')`로 함수를 불렀습니다. 시간대는 Chrome 설정으로 고정했습니다.

서울 (지금 = 2026-10-08 12:00 서울 = 03:00 UTC)

서버 시각 (UTC)	결과	확인한 것
`…03:00:00`, `…02:59:30`, `…03:05:00`	`Just now` ×3	같은 시각, 30초 전, 미래 5분
`…02:59:00`	`1 minute ago`	단수형
`…02:48:00`, `…02:47:59.500000`, `…02:48:00Z`, `…11:48:00+09:00`	`12 minutes ago` ×4	서버 형식(소수 6자리), `Z`, `+09:00`이 모두 같게 나옵니다

서버 시각 (UTC)	결과	확인한 것
`…02:48:00.123000`	`11 minutes ago`	11분 59.877초이므로 버림합니다
`…02:00:00`, `10-07T15:30:00`	`1 hour ago`, `11 hours ago`	서울로는 오늘 00:30입니다
`10-07T02:00:00`, `10-06T15:00:00`	`Yesterday` ×2	25시간 전, 서울 기준 어제 자정
`10-06T14:59:00`	`2 days ago`	서울 기준 그저께 23:59입니다 (1분 차이로 이틀 전)
`10-01T03:00:00`, `09-30T03:00:00`	`7 days ago`, `Sep 30`	7일까지는 "days ago", 8일부터는 날짜입니다
`2025-12-31T03:00:00`	`Dec 31, 2025`	다른 해

뉴욕 (서머타임이 끝나는 11월 1일 사이)

경우	결과
10월 31일 정오(EDT) → 11월 3일 정오(EST): 사이에 25시간짜리 하루가 있음	`3 days ago`
23.5시간 전 (날짜로는 어제)	`23 hours ago`
`now` 생략 (방금 시각)	`Just now`

처음 돌렸을 때 1개가 기대와 달랐는데, 함수가 아니라 **시험의 기대값**이 틀린 것이었습니다 (`02:48:00.123`은 11분 59.877초 전이므로 `11 minutes ago`가 맞습니다). 기대값을 고치고, 12분이 되는 서버 형식의 시각 (`02:47:59.500000`)을 따로 넣었습니다.